

🏗️ プロジェクト全体像 - アーキテクチャ図解ガイド

このプロジェクトの全体を図解で理解する

📖 目次

1. システムの全体構成 2. 各コンポーネントの役割 3. Phase 1~19 の実装ロードマップ 4. データフロー 5. 実装ス tack

🎯 システムの全体構成

```
mermaid graph TB
    User["👤 ユーザー"] -->|質問を投げかけ| Dashboard["📊 ダッシュボード"]
    User -->|管理・設定| Admin["⚙️ 管理画面"]
    Dashboard -->|推論リクエスト| API["🔌 API"]
    Admin -->|設定変更| API
    API -->|クエリ処理| Agent["🤖 自立型RAG Agent"]
    Agent -->|検索| RAG["📚 RAG 検索拡張生成"]
    Agent -->|評価| Scorer["⭐️ 自律性スコアラー"]
    Scorer -->|改善| FineTune["🎯 ファインチューニングエンジン"]
    RAG -->|知識ベース検索| KB["📖 知識ベース Corpus"]
    FineTune -->|パラメータ調整| Model["🧠 LLM モデル"]
    Agent -->|推論| Model
    Model -->|スコア計算| Feedback["📈 フィードバック"]
    Feedback -->|学習データ| FineTune
    Model -->|出力| Response["💬 回答"]
    Response -->|表示| Dashboard
    style User fill:#ffffcc
    style Dashboard fill:#ccffff
    style Agent fill:#ffcccc
    style Model fill:#ccccff
    style RAG fill:#ccffcc
    style KB fill:#ccffcc
```

🔧 各コンポーネントの役割

1. 🤖 自立型 RAG Agent

自分で判断して行動する AI エージェント

```
mermaid graph LR
    A["ユーザーの質問"] -->|受け取る| B["RAG Agent"]
    B -->|判断| C["必要な行動は？"]
    C -->|知識が必要| D["知識ベース検索"]
    C -->|改善が必要| E["ファインチューニング"]
    C -->|品質評価が必要| F["スコアリング"]
    D --> G["回答生成"]
    E --> G
    F --> G
    G -->|返す| H["最終回答"]
    style B fill:#ffcccc
    style C fill:#ffeeee
    style H fill:#ccffcc
```

できることの例： - 質問内容に応じて最適な知識ベースを検索 - 回答の品質が低い場合は、モデルを自動改善 - 自分の性能を評価して、次の学習に反映

2. 📚 RAG（検索拡張生成）

外部知識を使って高精度な回答を生成

質問：「2024年の日本の首相は？」

従来のLLM: 学習データは2023年まで → 知らない

RAG付きLLM: 知識ベースから「2024年首相」を検索 → 「岸田文雄」を発見 → 回答に統合 → 「2024年の首相は岸田文雄です」

mermaid graph LR; Q["質問"] -->|埋め込みベクトル化| Embed["Embedding"]; Embed -->|ベクトル検索| KB["知識ベース"]

索引["KB"] -->|関連文書取得| Docs["関連文書"]; Docs -->|プロンプトに挿入| Model["LLM"]; Q -->|コンテキスト追加| Model; Model -->|生成| Answer["回答"]; style Embed fill:#ffeeee style KB fill:#ffffcc style Model fill:#ccffff style Answer fill:#ccffcc

3. ★ 自律性スコアラー

AIの自立性（自分で判断できる度合い）を測定

例：AIが「この情報は信頼できない」と判断する → 判断の根拠を説明できる → 自律性スコア UP！

mermaid graph TB; Action["AIの行動"] -->|測定| Autonomy["自律性スコア"]

9つの判定基準["Autonomy"] -->|1| C1["計画性"]

目標を立てるか["Autonomy"] -->|2| C2["学習性"]

過去から学ぶか["Autonomy"] -->|3| C3["適応性"]

環境に対応するか["Autonomy"] -->|4| C4["判断性"]

自分で判断するか["Autonomy"] -->|5| C5["説明性"]

理由を説明できるか["C1"] --> Score["最終スコア"]

0~100["C2"] --> Score; C3 --> Score; C4 --> Score; C5 --> Score; Score -->|フィードバック| Learn["改善ループ"]

Autonomy fill:#ffffcc style Score fill:#ffcccc style Learn fill:#ccffff

4. 🎯 ファインチューニング

モデルを特定のタスクに合わせて調整

元のモデル（汎用） ↓ [調整用データ: 医療QA 1000例] ↓ 医療特化モデル（精度向上）

mermaid graph LR; Model1["事前学習済みモデル"] -->|調整データ投入| Process["ファインチューニング"]

汎用["Model1"] -->|調整データ投入| Process; Process -->|学習| Data["医療QAデータ"]

Process -->|最適化| Model2["医療特化モデル"]

精度向上["Model2"] style Model1 fill:#ffcccc style Process fill:#ffeeee style Model2 fill:#ccffcc style Data fill:#ffffcc

5. 📖 知識ベース（Corpus）

質問に対して関連情報を提供するデータストア

mermaid graph TB; Sources["情報源"] -->|取り込み| OCR["OCR処理"]

画像→テキスト["Web"] -->|スクレイピング| Clean["クリーニング"]

ノイズ除去["DB"] -->|抽出| Process["前処理"]

OCR -->|チャンク化| Chunk["段落分割"]

512トークン["Clean"] -->|チャンク化| Chunk; Chunk -->|チャンク化| Chunk; Chunk -->|埋め込み化| Embed["Vector化"]

Embed -->|インデックス化| Index["検索インデックス"]

高速検索対応["Index"] -->|保存| Storage["Corpus"]

永続化["style Index fill:#ffffcc style Embed fill:#ffeeee style Storage fill:#ccffcc"]

📋 Phase 1～19 の実装ロードマップ

```
graph LR
    subgraph Phase_1_3 [Phase 1-3 基盤]
        P1[p1, 2026-01-01, 30d] --> P2[p2, after p1, 30d]
        P2 --> P3[p3, after p2, 30d]
    end
    subgraph Phase_4_6 [Phase 4-6 推論]
        P4[p4, after p3, 30d] --> P5[p5, after p4, 30d]
        P5 --> P6[p6, after p5, 30d]
    end
    subgraph Phase_7_9 [Phase 7-9 学習]
        P7[p7, after p6, 30d] --> P8[p8, after p7, 30d]
        P8 --> P9[p9, after p8, 30d]
    end
    subgraph Phase_10_12 [Phase 10-12 最適化]
        P10[p10, after p9, 30d] --> P11[p11, after p10, 30d]
        P11 --> P12[p12, after p11, 30d]
    end
    subgraph Phase_13_15 [Phase 13-15 運用]
        P13[p13, after p12, 30d] --> P14[p14, after p13, 30d]
        P14 --> P15[p15, after p14, 30d]
    end
    subgraph Phase_16_19 [Phase 16-19 本番]
        P16[p16, after p15, 30d] --> P17[p17, after p16, 30d]
        P17 --> P18[p18, after p17, 30d]
        P18 --> P19[p19, after p18, 30d]
    end
```

各フェーズの概要

| フェーズ | テーマ | 実装内容 | |-----|-----|-----| | **1-3** | **基盤構築** | スケジューラー、ロールバック、バージョン管理 | | **4-6** | **推論システム** | バッチ処理、キャッシング、レイテンシ最適化 | | **7-9** | **学習機構** | オンライン学習、勾配累積、メモリ効率化 | | **10-12** | **最適化** | 言語能力改善、マルチモーダル対応 | | **13-15** | **運用体制** | 監視、デプロイ、自動スケーリング | | **16-19** | **本番運用** | セキュリティ、コンプライアンス、本番環境 |

🔄 データフロー

学習～推論の完全フロー

```
graph LR
    User --> Dashboard
    Dashboard --> Agent
    Agent --> RAG
    RAG --> Agent
    Agent --> Model
    Model --> Model
    Model --> Agent
    Agent --> Scorer
    Scorer --> Agent
    Agent --> FineTune
    FineTune --> Model
    Model --> Dashboard
    Dashboard --> User
```

🔧 実装スタック

技術選択

```
graph TD
    LLM[LLM] --> PyTorch[PyTorch]
    LLM --> HF[Hugging Face]
    LLM --> KB[知識ベース]
    KB --> Milvus[Milvus]
    KB --> NLTK[NLTK/Spacy]
    API[API/インターフェース] --> FastAPI[FastAPI]
    UI[UI] --> Streamlit[Streamlit]
    Data[データ管理] --> Redis[Redis]
    Data --> PostgreSQL[PostgreSQL]
    Infra[インフラ] --> Docker[Docker]
    Infra --> K8s[Kubernetes]
```

スケーリング"] style LLM fill:#ffcccc style KB fill:#ccffcc style API fill:#ccccff style Data fill:#ffffcc style Infra fill:#ffeeee

、

ファイル構成の全体像

、 /home/abemc/project_root/ |— 🤖 コアシステム | |— autonomous ragagent.py ← RAG Agent 実装 | |— ragagentconfig.py ← 設定ファイル | |— ragagentsidebar.py ← UIコンポーネント | |— 📊 ダッシュボード | |— dashboard_app.py ← Streamlit UI | |— app.py ← メインアプリ | |— 🧠 モデル関連 | |— finetunedmodel/ ← ファインチューニング済みモデル | |— embeddings/ ← 埋め込みモデル | |— checkpoints/ ← トレーニングチェックポイント | |— 📖 知識ベース | |— corpus/ ← テキスト索引 | |— raw_pdfs/ ← 元のPDFファイル | |— data/ ← 構造化データ | |— 🛠️ 管理・運用 | |— manage_kb.py ← 知識ベース管理 | |— docs_manager.py ← ドキュメント管理 | |— main.py ← エントリーポイント | |— 📝 学習・評価 | |— src/evaluation/ ← 評価フレームワーク | |— logs/ ← 実行ログ | |— checkpoints/ ← チェックポイント | |— 📄 ドキュメント |— docs/ ← この資料を含む 、

実装の流れ

初心者が学習する際の推奨順序：

、 mermaid graph TD A["この資料を読了
全体像理解"] -->|☒ 確認| B["段階2:
環境構築"] B -->|実行| C["サンプルコード実行
動作確認"] C -->|理解| D["各コンポーネント
詳細学習"] D -->|実装| E["簡単なタスク実装
例: RAG検索"] E -->|拡張| F["本プロジェクトの
コード読解"] F -->|応用| G["独自機能実装
カスタマイズ"] style A fill:#ffffcc style B fill:#ffcccc style C fill:#ccffcc style D fill:#ccccff style E fill:#ffeeee style F fill:#eeeeee style G fill:#ccffff 、

☒ 理解度チェック

- [] システムの5つの主要コンポーネント（Agent・RAG・Scorer・FineTune・KB）の役割が説明できる - [] RAG が「検索」+「生成」の組み合わせだと理解した - [] 自律性スコアラーが何を測定しているか説明できる - [] ファインチューニングとは何か説明できる - [] 知識ベース（Corpus）の構築フローが理解できた - [] 19のPhaseがどのような順序で実装されるか理解した - [] 技術スタック（PyTorch・Hugging Face・FastAPI など）の役割が説明できる

次のステップ

☒ このガイドを読み終わったら → [段階2：環境構築と実装ハンズオン](#) へ

次のガイドでは、実際にコードを動かしながら、LLMの基本をPythonで実装します。

質問やフィードバック: Issue を作成するか、ドキュメント管理者に連絡してください